

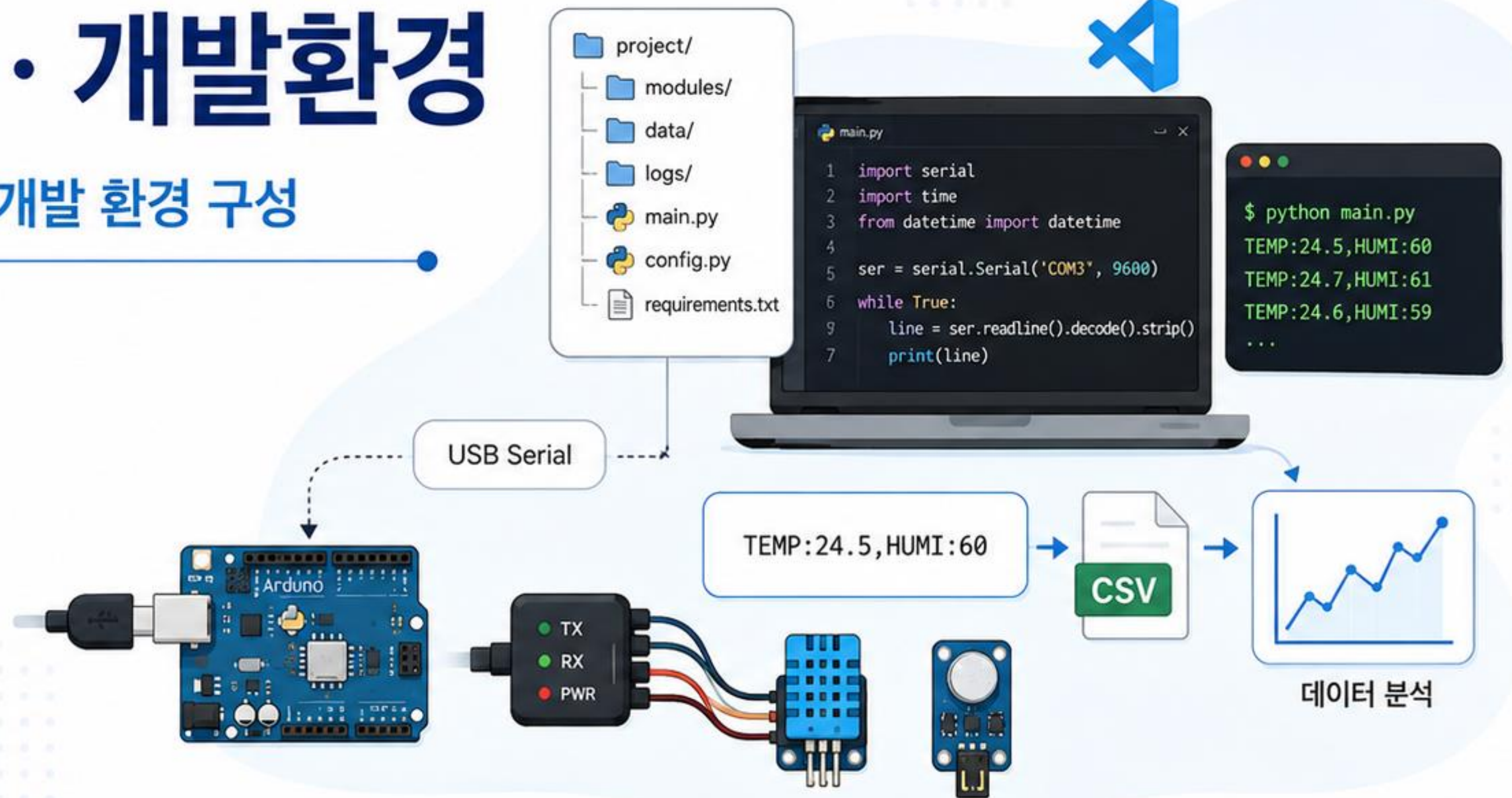
Day 2

모듈 · 패키지 · 개발환경

Python 프로젝트 구조와 IoT 개발 환경 구성

임베디드 · IoT 개발자 과정

- 📦 모듈과 패키지 이해
- >_ 개발 환경 구성 (pip, venv)
- 🔗 시리얼 통신 (pyserial)
- 📊 데이터 처리 및 저장



오늘의 학습 목표

Day 2에서 배우게 될 핵심 내용을 한눈에 정리합니다.

- 01  Python 모듈 구조 이해
- 02  표준 라이브러리 활용
- 03  pip와 외부 패키지 설치
- 04  가상환경 구성
- 05  Arduino와 Serial 통신 실습



Python 프로젝트 구조부터 Serial 데이터 처리까지 단계적으로 학습합니다.



Day 1에서는 Python 기초 문법과 기본 자료구조를 학습했습니다. 핵심 내용을 다시 한번 정리해 봅시다.



Python 특징

간결한 문법, 다양한 라이브러리, 크로스 플랫폼, 빠른 개발 생산성



변수와 자료형

int, float, str, bool 타입과 변수 선언, 자료형 변환



연산자

산술 연산자, 비교 연산자, 논리 연산자, 할당 연산자



조건문과 반복문

if, elif, else, while, for 반복문과 range() 활용



함수

함수 정의, 매개변수, 반환값, 영역(scope)



자료구조

list, tuple, set, dict의 생성, 접근, 수정, 순회 방법

간단한 코드 예시

```
num = 10
text = "Python"
arr = [1, 2, 3]

if num > 5:
    for i in range(3):
        print(i, text)
```

출력 결과

```
0 Python
1 Python
2 Python
```

자료구조 한눈에 보기

list (리스트)

[1, 2, 3, 4]

1 2 3 4

↓ +

순서가 있고,
변경 가능

tuple (튜플)

(1, 2, 3, 4)

1 2 3 4

순서가 있고,
변경 불가

set (집합)

{1, 2, 3, 4}

1 2 3 4

순서 없고,
중복 허용 안 함

dict (딕셔너리)

{'a': 1, 'b': 2}

a	1
b	2

키-값 쌍으로
저장



오늘은 코드를 여러 파일로 나누고, 외부 라이브러리를 활용하여 더 효율적인 프로그램을 작성하는 방법을 배웁니다!

코드를 모듈(파일) 단위로 나누면 재사용성과 유지보수성이 높아지고, 협업과 확장에 유리합니다.



Point

프로그램이 커질수록 한 파일의 코드는 복잡해지고 수정이 어려워집니다. 기능별로 나누어 관리하세요!

모듈 없이 한 파일에 모두 작성하면?

main.py

```
# --- 센서 관련 코드 ---
def read_sensor();
    pass

# --- 데이터 처리 코드 ---
def process_data(data);
    pass

# --- 파일 저장 코드 ---
def save_to_file(data);
    pass

# --- 화면 출력 코드 ---
def print_result(data);
    pass

# --- 메인 실행 ---
if __name__ == "__main__":
    ...
```



- ✗ 코드가 길어지고 복잡해짐
- ✗ 수정 시 다른 부분에 영향
- ✗ 재사용이 어려움
- ✗ 역할이 섞여 유지보수 어려움
- ✗ 협업 시 충돌이 잦음



작은 프로그램은 괜찮지만,
프로젝트가 커질수록 문제가 커집니다!

모듈로 나누어 작성하면?

sensor_project/

sensor.py	# 센서 읽기
process.py	# 데이터 처리
storage.py	# 파일 저장
display.py	# 화면 출력
main.py	# 메인 실행

- ✓ 역할과 기능이 명확해짐
- ✓ 수정과 테스트가 쉬워짐
- ✓ 다른 프로젝트에서 재사용 가능
- ✓ 유지보수와 확장이 쉬움
- ✓ 협업 시 각자 맡은 부분 개발



코드가 깔끔해지고,
확장과 재사용이 쉬워집니다!

Arduino 라이브러리 vs Python 모듈



Arduino 라이브러리

- .h, .cpp 파일로 구성
- 클래스와 함수 제공
- 재사용과 추상화 지원
- 예: LiquidCrystal, DHT



Python 모듈

- .py 파일로 구성
- 함수, 변수, 클래스 제공
- 재사용과 코드 분리 지원
- 예: math, datetime, mymodule

★ 핵심 정리

- 모듈은 '기능 단위의 코드 파일'입니다.
- 모듈을 사용하면 코드가 깔끔해지고, 유지보수와 확장이 쉬워집니다.
- Python도 Arduino처럼 모듈과 패키지를 활용해 프로젝트를 구성합니다.

5 Python 모듈(Module)이란?

모듈은 함수, 변수, 클래스 등을 모아둔 파이썬 파일(.py)입니다. 다른 파일에서 import하여 사용할 수 있습니다.



Point

Python은 모든 .py 파일을 모듈로 사용할 수 있습니다.
모듈을 활용하면 코드 재사용과 관리가 쉬워집니다.

1. 모듈의 구성

모듈은 변수, 함수, 클래스 등을 포함할 수 있습니다.

mymath.py

```
# 변수
PI = 3.14159

# 함수
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

# 클래스
class Calculator:
    def __init__(self):
        self.result = 0
    def mul(self, a, b):
        self.result = a * b
        return self.result
```

변수

모듈 내부에서 사용되는 값

함수

특정 기능을 수행하는 코드
묶음

클래스

객체지향 프로그래밍을 위한
설계도(클래스)



Tip

모듈 파일명은 파이썬 식별자 규칙을 따라야 합니다.
예: mymath.py, sensor.py (공백, 특수문자 사용 불가)

2. 다른 파일에서 모듈 사용하기

모듈을 import하여 모듈 내부의 기능을 사용할 수 있습니다.

main.py

```
import mymath

print(mymath.PI)           # 3.14159
print(mymath.add(3, 5))    # 8
print(mymath.sub(10, 4))   # 6

calc = mymath.Calculator()
print(calc.mul(3, 4))      # 12
```

main.py
(사용하는 파일)

import

모듈의 기능을 가져와 사용

mymath.py
(모듈 파일)

모듈의 특징

- ✓ 재사용 가능
- ✓ 코드 분리로 가독성 향상
- ✓ 프로젝트 규모 확장에 유리

3. 모듈 사용의 장점

모듈을 사용하면 개발 효율과 유지보수성이 높아집니다.



코드 재사용

한 번 만든 모듈을 여러 프로젝트에서
반복 사용할 수 있습니다.



기능 분리

기능별로 코드를 나누어 작성하여
이해하기 쉬워집니다.



유지보수 편리

수정이 필요한 부분만 모듈에서 변경하면
전체 프로그램에 적용됩니다.



협업에 유리

팀원이 모듈을 나누어 개발하고
합치기 쉬워집니다.



주의사항

- 모듈 파일명이 겹치지 않도록 주의하세요.
- 순환 import(서로 import) 구조는 피하세요.

간단한 수학 기능을 제공하는 모듈을 만들고, 다른 파일에서 사용해 봅시다.



Point

모듈은 .py 파일 하나로 만들 수 있습니다.
기능별로 분리하면 코드 관리가 쉬워집니다!

1. 모듈 파일 작성 : mymath.py

mymath.py

```
# 상수(변수)
PI = 3.14

# 함수: 두 수의 합
def add(a, b):
    return a + b

# 함수: 두 수의 차
def sub(a, b):
    return a - b

# 함수: 원의 넓이
def circle_area(r):
    return PI * r * r
```

상수(변수)

모듈에서 사용할 값을 정의합니다.

함수

특정 기능을 수행하는 코드를 정의합니다.

여러 기능을 모아

하나의 파일(mymath.py)로 만듭니다.



2. 모듈 사용 : main.py

main.py

```
import mymath

print("파이값:", mymath.PI)
result1 = mymath.add(3, 5)
result2 = mymath.sub(10, 4)
area = mymath.circle_area(5)

print("3 + 5 =", result1)
print("10 - 4 =", result2)
print("반지름 5의 원 넓이:", area)
```

실행 결과

```
파이값: 3.14
3 + 5 = 8
10 - 4 = 6
반지름 5의 원 넓이: 78.5
```



main.py

(사용하는 파일)

import

모듈의 기능을
가져와 사용



mymath.py

(모듈 파일)

★ 핵심 정리

- 모듈은 함수, 변수(상수), 클래스를 모아둔 .py 파일입니다.
- 기능별로 모듈을 나누면 재사용성과 유지보수성이 높아집니다.
- 다른 파일에서 import하여 모듈의 기능을 사용할 수 있습니다.



실습 미션

1. mystring.py 모듈을 만들어 문자열 길이를 구하는 함수(len_str)와 특정 문자가 몇 번 나오는지 세는 함수(count_char)를 정의하세요.
2. main.py에서 해당 모듈을 import하여 함수들을 사용해 보세요!

파일 구조 예시

```
my_project/
├── mymath.py
└── main.py
```

Tip

파일 이름은 소문자와 언더스코어(_)를 사용하는 것이 권장됩니다!

7 import 사용법

모듈을 가져오는 다양한 방법을 이해하고 상황에 맞게 사용해 봅시다.

 **Point**

필요한 것만 명확하게 import하는 것이
코드 가독성과 유지보수에 도움이 됩니다!

1. import module

모듈 전체를 가져와서 사용합니다.

 mymath.py

```
PI = 3.14
def add(a, b):
    return a + b
def sub(a, b):
    return a - b
```

사용 예 (main.py)

```
import mymath
print(mymath.PI)
print(mymath.add(2, 3))
print(mymath.sub(5, 2))
```

실행 결과

```
3.14
5
3
```

✔ 특징

- 모듈명.함수명 형태로 사용
- 다른 이름과 충돌 위험이 적음

2. from module import function

모듈에서 필요한 함수(또는 변수)만 가져옵니다.

 mymath.py

```
PI = 3.14
def add(a, b):
    return a + b
def sub(a, b):
    return a - b
```

사용 예 (main.py)

```
from mymath import add, sub
print(add(2, 3))
print(sub(5, 2))
```

실행 결과

```
5
3
```

✔ 특징

- 필요한 함수(또는 변수)만 가져옴
- 모듈명 없이 바로 사용 가능

3. from module import *

모듈의 모든 함수, 변수, 클래스를 가져옵니다.

 mymath.py

```
PI = 3.14
def add(a, b):
    return a + b
def sub(a, b):
    return a - b
```

사용 예 (main.py)

```
from mymath import *
print(PI)
print(add(2, 3))
print(sub(5, 2))
```

실행 결과

```
3.14
5
3
```

⚠ 주의사항

- 코드가 길어질수록 어떤 이름이 어디서 왔는지 알기 어려움
- 다른 모듈과 이름 충돌 위험이 있음 (가급적 지양)

비교 요약

방법	예시	사용 방법	장점	단점	권장도
import module	import mymath	mymath.add()	충돌 위험 적음, 명확함	코드가 길어질 수 있음	★★★★☆
from module import function	from mymath import add	add()	코드 간결, 가독성 좋음	가져올 항목이 많으면 길어짐	★★★★★☆☆
from module import *	from mymath import *	add()	모든 항목 사용 가능	이름 충돌, 가독성 저하	★★★☆☆☆☆

 **권장 가이드**

- 프로젝트에서는 "from module import 함수명" 방식을 기본으로 사용하세요.
- 모듈의 역참을 명확히 하고, 필요한 것만 가져오는 습관을 들이세요.
- 팀 개발 시에는 import 규칙을 정해 일관되게 사용하세요.

__name__ 과 __main__

Python 모듈이 직접 실행될 때와 다른 모듈에서 import될 때의 차이를 이해해야 합니다.

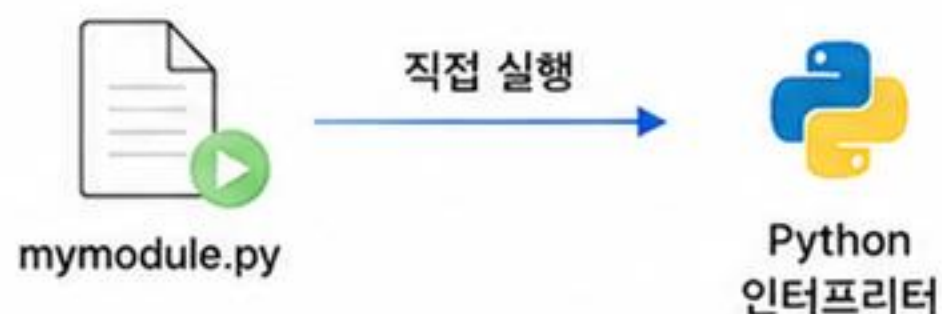


Point

__name__ 변수를 사용하면
모듈을 스크립트로 실행할 때만 특정 코드를 실행할 수 있습니다!

1. 모듈을 직접 실행할 때

Python 파일을 직접 실행하면
__name__의 값은 "__main__" 입니다.



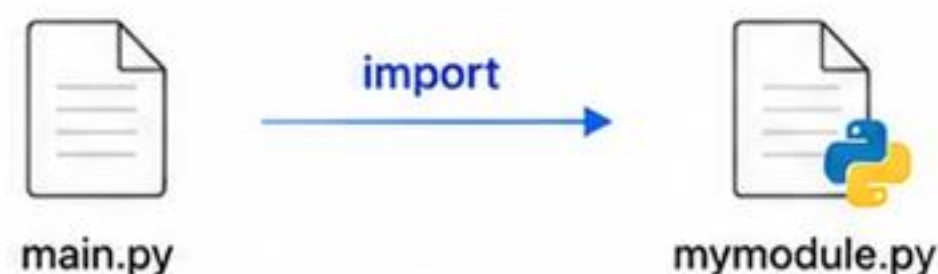
```
__name__ == "__main__"
( True )
```

if __name__ == "__main__" 블록이 실행됩니다.

```
$ python mymodule.py
모듈을 직접 실행했습니다!
```

2. 다른 모듈에서 import될 때

다른 Python 파일에서 import하면
__name__의 값은 **모듈의 이름**입니다.



```
__name__ == "mymodule"
( False )
```

if __name__ == "__main__" 블록은 실행되지 않습니다.

```
$ python main.py
(실행되는 출력 없음)
```

코드 예시 : mymodule.py

```
1 def hello():
2     print("안녕하세요! 모듈 안의 함수입니다.")
3
4 def main():
5     print("main() 함수가 실행되었습니다.")
6
7 if __name__ == "__main__":
8     print("이 파일은 직접 실행되었습니다.")
9     main()
10 else:
11     print(f"이 파일은 import되었습니다. (__name__ = {__name__})")
```

실행 결과

직접 실행
(python mymodule.py)

```
이 파일은 직접 실행되었습니다.
main() 함수가 실행되었습니다.
```

import하여 실행
(다른 파일에서 import)

```
이 파일은 import되었습니다.
(__name__ = mymodule)
```

☆ 핵심 정리

- __name__ 변수는 현재 모듈의 이름을 담고 있습니다.
- 직접 실행하면 "__main__" 이고, import되면 모듈 이름입니다.
- if __name__ == "__main__": 블록은 직접 실행 시에만 동작합니다.

① 주요 활용 예

- 테스트 코드 작성 (직접 실행 시에만 테스트 수행)
- 예제 실행 코드 작성 (사용 예시 제공)
- 모듈을 라이브러리로 제공할 때 불필요한 실행 방지



실습 미션

mymodule.py 파일을 만들고, 위의 코드를 직접 실행해 보세요.
그리고 다른 파일(main.py)에서 import하여 사용해 보세요.
__name__의 값이 어떻게 달라지는지 확인해 봅시다!

패키지는 관련된 여러 모듈을 폴더(디렉터리) 단위로 묶어 체계적으로 관리하는 구조입니다.



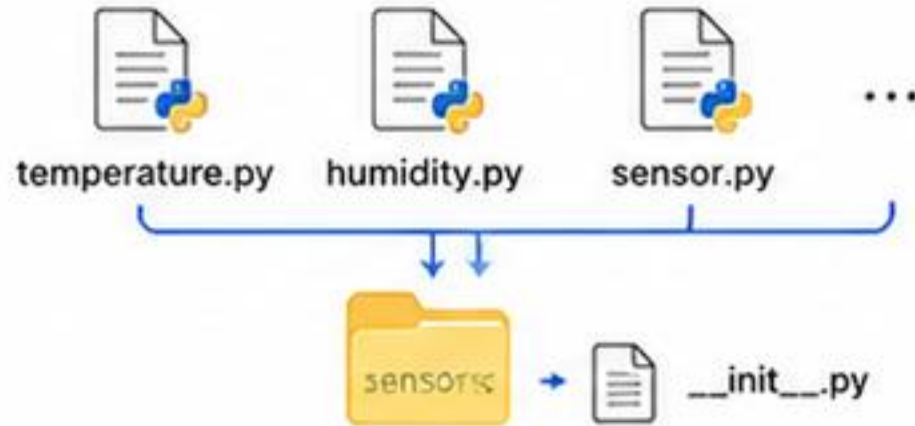
Point

패키지를 사용하면 기능을 체계적으로 분리하고, 이름 충돌을 방지하며, 재사용과 유지보수가 쉬워집니다!

1. 패키지 개념

패키지 = 모듈들의 집합

하나의 주제를 가진 여러 모듈을
폴더로 묶고, `__init__.py` 파일을
넣어 패키지로 만듭니다.



장점

- 관련 기능을 체계적으로 관리
- 이름 충돌 방지
- 코드 재사용성 향상
- 프로젝트 구조가 명확해짐

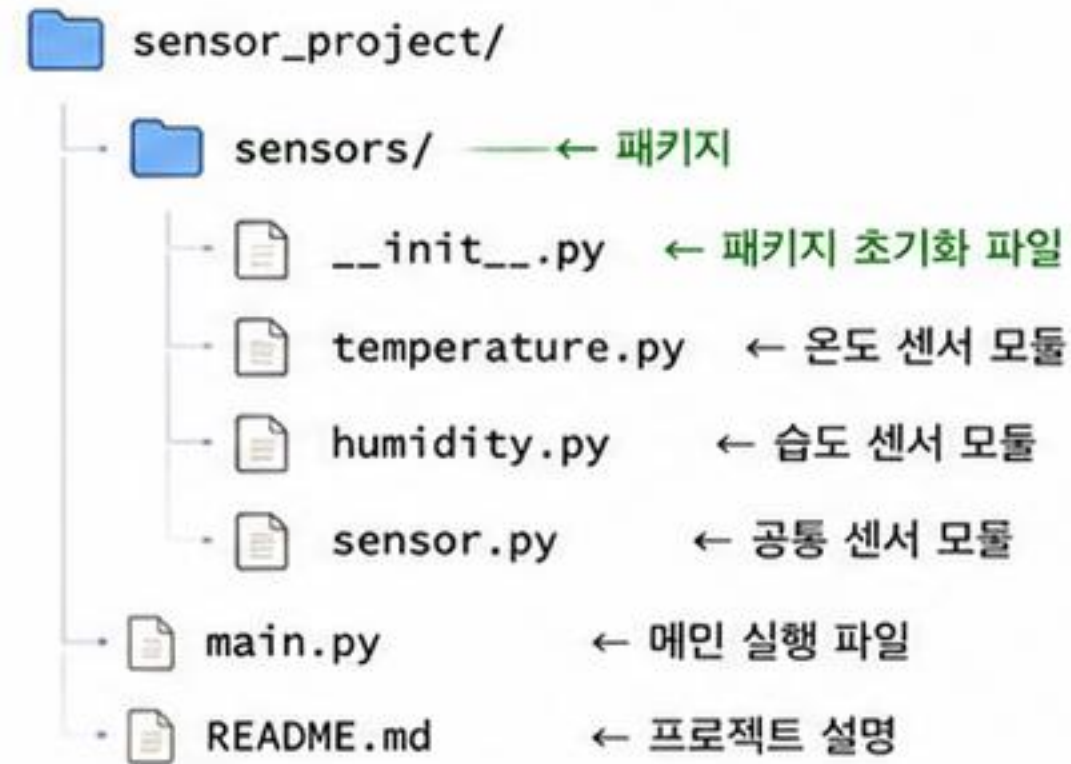


핵심 정리

- 패키지는 여러 모듈을 폴더로 묶은 구조입니다.
- 패키지 폴더에는 `__init__.py` 파일이 있어야 합니다.
- 패키지를 사용하면 코드를 체계적으로 관리하고 재사용하기 쉬워집니다.

2. 패키지 구조 예시

예시: 센서 관련 기능을 묶은 패키지



`__init__.py` 파일

- 패키지임을 알리는 역할
- 패키지 초기화 코드 작성 가능 (선택)
- 비워두어도 패키지로 동작 (Python 3.3+)



실습 미션

1. sensors 패키지 폴더를 만들고, `__init__.py` 파일을 추가하세요.
2. `temperature.py`, `humidity.py` 모듈을 작성하세요.
3. `main.py`에서 패키지의 모듈을 import하여 사용해 보세요.

3. 패키지 사용 방법

다른 파일에서 패키지의 모듈을 사용합니다.

예시 1: 전체 모듈 import

```
import sensors.temperature as temp
import sensors.humidity as hum

print(temp.read())      # 온도 읽기
print(hum.read())       # 습도 읽기
```

예시 2: 필요한 함수만 import

```
from sensors.temperature import read
from sensors.humidity import read

print(read())           # 온도 읽기
print(read())           # 습도 읽기
```



참고

패키지 내부의 모듈도 다른 모듈을 import할 수 있습니다.
상대 import (from . import 모듈명) 도 가능합니다.



Tip

- 패키지 이름은 소문자, 여러 단어는 언더스코어(_)를 사용하세요.
- 프로젝트가 커질수록 패키지 구조가 매우 중요해집니다!

10

실무형 Python 프로젝트 구조

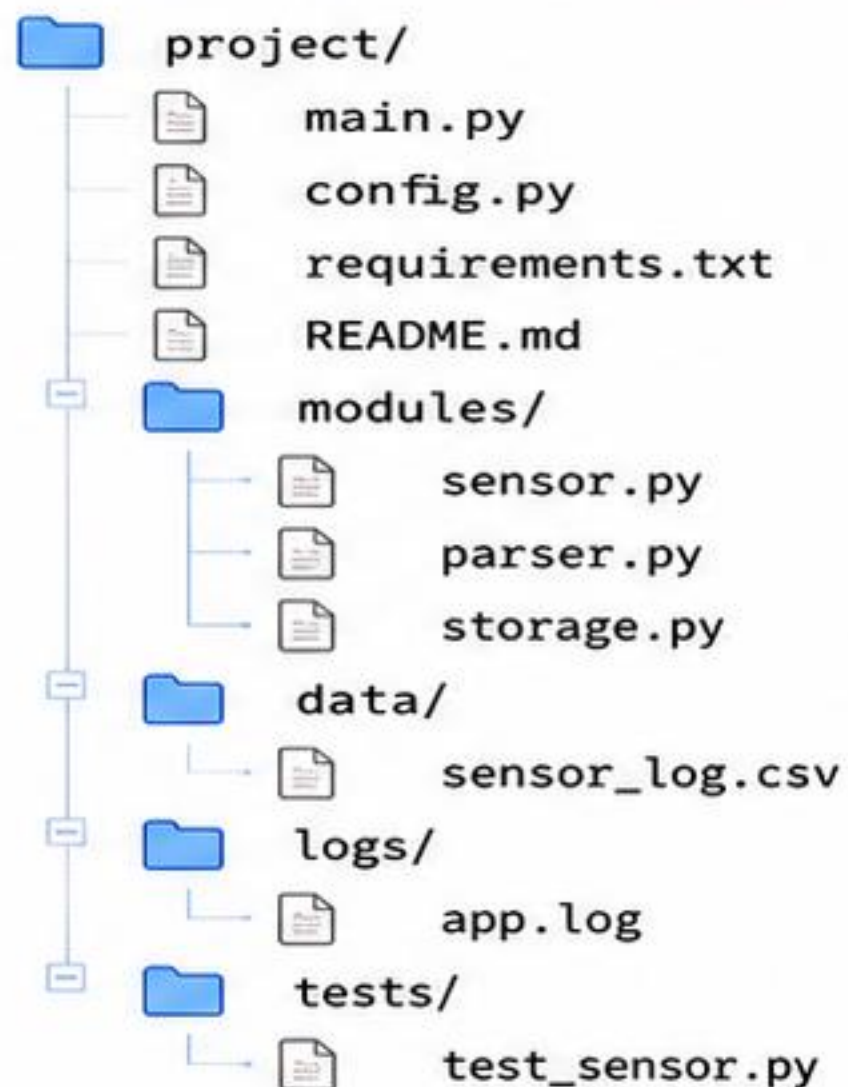
프로젝트가 커질수록 파일과 폴더를 체계적으로 구성하는 것이 중요합니다.



Point

프로젝트 구조가 명확하면
유지보수, 협업, 테스트, 배포가 쉬워집니다.

1. 예시 프로젝트 구조



2. 각 구성 요소의 역할



main.py

: 프로그램 시작점



config.py

: 설정값 관리



requirements.txt

: 필요한 패키지 목록



modules/

: 기능별 코드 분리



data/ · logs/

: 데이터와 실행 기록 저장



tests/

: 기능 검증 코드



main.py

프로그램 시작



modules/

기능 모듈 호출

data 저장 /
logs 기록

데이터 저장 및 기록



tests/

기능 검증



핵심 정리

- 한 파일에 모든 코드를 넣지 않는다.
- 기능별로 모듈과 폴더를 나눈다.
- 프로젝트 규모가 커질수록 구조가 더 중요하다.



실습 미션

1. project 폴더 만들기
2. modules, data, logs 폴더 만들기
3. main.py 와 requirements.txt 추가하기

직접 나만의 패키지를 만들어 보고, 다른 파일에서 import하여 사용하는 방법을 실습해 봅니다.



Point

직접 패키지를 만들면
파이썬의 모듈 관리 원리를 더 깊이 이해할 수 있습니다!

1. 패키지 구조 만들기

math_utils 라는 패키지를 만들고,
그 안에 여러 모듈을 추가합니다.



i 폴더/파일 역할

- math_utils/ : 패키지 폴더
- __init__.py : 패키지 초기화 (비워둬도 OK)
- calc.py : 계산 관련 함수 모듈
- stats.py : 통계 관련 함수 모듈
- main.py : 패키지를 사용하는 메인 파일

🔗 핵심 정리

- 패키지는 관련 모듈을 폴더로 묶어 관리하는 방법입니다.
- __init__.py 파일이 있어야 패키지로 인식됩니다.
- import 문을 사용해 다른 파일에서 패키지의 모듈을 사용할 수 있습니다.

2. 모듈 코드 작성

math_utils/calc.py

```
def add(a, b):
    return a + b
```

```
def mul(a, b):
    return a * b
```

math_utils/stats.py

```
def mean(nums):
    return sum(nums) / len(nums)
```

```
def maximum(nums):
    return max(nums)
```

math_utils/__init__.py

```
# 패키지 초기화 파일 (비워둬도 됩니다)
# 필요하면 __all__ 등을 정의할 수도 있습니다.
```

```
__all__ = ['calc', 'stats']
```

☆ 참고

__all__ 리스트를 정의하면
from math_utils import * 할 때 지정한 모듈만 import됩니다.

🚀 실습 미션

1. 문자열 처리 함수를 모아 string_utils 패키지를 만들어 보세요.
2. 내 패키지에 __all__을 정의하고 * import의 결과를 확인해 보세요.
3. 패키지를 친구와 공유하고 사용해 보세요!

3. 패키지 사용하기 (main.py)

main.py

```
# 방법 1: 전체 모듈 import
from math_utils import calc, stats
print(calc.add(3, 5))
print(calc.mul(4, 2))
print(stats.mean([1, 2, 3, 4, 5]))
print(stats.maximum([1, 2, 3, 4, 5]))
```

```
# 방법 2: 필요한 함수만 import
from math_utils.calc import add
from math_utils.stats import mean
print(add(10, 20))
print(mean([10, 20, 30]))
```

실행 결과

```
8
8
3.0
5
30
20.0
```



Tip

- 패키지 이름은 소문자_언더스코어(_)를 사용하는 것이 좋습니다.
- 계층 구조가 깊어도 import로 쉽게 사용할 수 있습니다.
- 큰 프로젝트일수록 패키지 구조가 중요합니다!

12 패키지 활용 정리 및 프로젝트 미션

지금까지 배운 패키지 개념과 사용법을 정리하고, 간단한 프로젝트를 만들어 실력을 확인해 봅시다.



Point

패키지를 잘 활용하면 코드를 구조적으로 만들고, 재사용과 유지보수를 쉽게 할 수 있습니다!

1. 핵심 정리



패키지란?

관련 모듈들을 폴더로 묶고, `__init__.py` 파일을 통해 하나의 패키지로 만드는 것



import 방법

`import` 패키지.모듈
`from` 패키지.모듈 `import` 함수



패키지의 장점

- 코드 구조가 명확해짐
- 재사용과 유지보수가 쉬워짐
- 팀 개발에 유리함



기억할 키워드

`__init__.py`, `import`, `from import`, 패키지 구조, 모듈



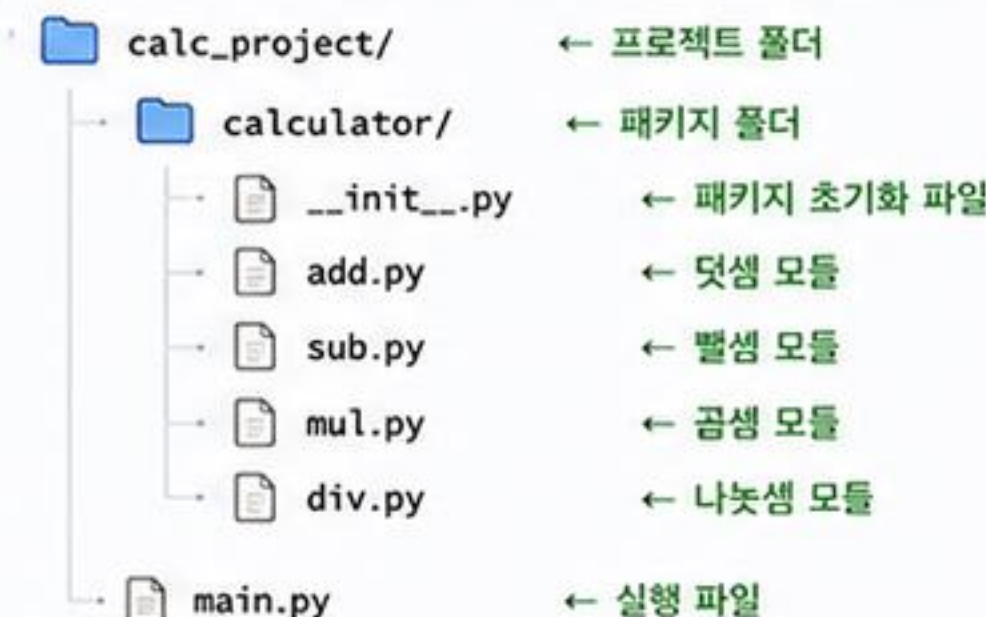
핵심 정리

- 패키지는 여러 모듈을 체계적으로 관리하는 방법입니다.
- `import`와 `from import`를 적절히 사용하면 더 편리합니다.
- 좋은 패키지 구조는 협업과 유지보수에 큰 도움이 됩니다.

2. 프로젝트 미션 : 간단한 계산기 패키지 만들기

계산기 기능(덧셈, 뺄셈, 곱셈, 나눗셈)을 제공하는 패키지를 만들어 사용해 봅시다.

폴더 구조 예시



☆ 미션 목표 목표

1. calculator 패키지를 만들고, 각 연산을 하는 모듈 적성
2. `__init__.py` 파일에서 필요한 함수들을 `import`
3. `main.py`에서 패키지를 `import`하여 계산기 기능 사용



실습 미션

1. 계산기 패키지(calc_project)를 직접 만들어 보세요.
2. 다른 연산(거듭제곱, 나머지 등)을 추가해 확장해 보세요.
3. 패키지를 다른 프로젝트에서 `import`하여 사용해 보세요.

3. 예시 코드 및 실행 결과

main.py

```
from calculator import add, sub, mul, div

print("3 + 4 =", add.add(3, 4))
print("7 - 2 =", sub.sub(7, 2))
print("3 * 5 =", mul.mul(3, 5))
print("10 / 2 =", div.div(10, 2))
```

add.py

```
def add(a, b):
    return a + b
```

mul.py

```
def mul(a, b):
    return a * b
```

__init__.py

```
# 필요한 함수들을 패키지에서 바로 사용할 수 있도록 import
from .add import add
from .sub import sub
from .mul import mul
from .div import div
```

sub.py

```
def sub(a, b):
    return a - b
```

div.py

```
def div(a, b):
    return a / b
```

실행 결과

```
3 + 4 = 7
7 - 2 = 5
3 * 5 = 15
10 / 2 = 5.0
```

✓ 확인해 보세요!

- 패키지 구조가 올바른지
- `__init__.py` 파일이 있는지
- 모듈 이름과 함수 이름이 일치하는지



도전 과제

- 패키지에 예외 처리 기능을 추가해 보세요.
- 사용자 입력을 받아 계산하는 프로그램을 만들어 보세요.
- 패키지를 `pip`로 배포하는 방법도 찾아보세요!



표준 라이브러리 외에 외부에서 제공하는 패키지를 설치하고, 내 프로젝트에서 사용해 봅니다.



Point

외부 패키지를 활용하면 복잡한 기능을
쉽게 구현할 수 있고, 개발 속도를 높일 수 있습니다!

1. 외부 패키지란?



Python 공식 문서에 포함되지 않은
수많은 개발자들이 만든 유용한 패키지

대표적인 외부 패키지 예시

- requests : HTTP 요청 처리
- numpy : 수치 연산, 배열 처리
- pandas : 데이터 분석
- matplotlib : 데이터 시각화
- BeautifulSoup4 : 웹 크롤링
- scipy : 과학 계산

⚠ 주의사항

- 신뢰할 수 있는 패키지인지 확인하고 사용
- 버전 호환성을 확인
- 가상환경에서 설치하는 것을 권장

🔍 패키지 정보 확인

```
$ pip show 패키지명 # 상세 정보 확인
$ pip list           # 설치된 패키지 목록
$ pip list --outdated # 업데이트 가능한 패키지 확인
```

2. 패키지 설치 방법 (pip)

pip는 Python 패키지 관리 도구입니다.

명령어 예시

```
$ pip install 패키지명 # 최신 버전 설치
$ pip install 패키지명==버전 # 특정 버전 설치
$ pip install --upgrade 패키지명 # 업그레이드
$ pip uninstall 패키지명 # 삭제
$ pip list # 설치된 패키지 목록
$ pip show 패키지명 # 패키지 정보 확인
```

가상환경에서 설치 (권장)

```
# 가상환경 활성화 후
$ pip install 패키지명

# 예시
$ pip install requests
```



requirements.txt 사용

프로젝트에 필요한 패키지 목록을 파일로 관리할 수 있습니다.

```
$ pip freeze > requirements.txt # 현재 환경 저장
$ pip install -r requirements.txt # 목록 기반 설치
```

3. 설치한 패키지 사용하기

설치한 패키지를 import하여 사용합니다.

example.py

```
1 import requests
2
3 url = "https://api.github.com"
4 response = requests.get(url)
5
6 print("상태 코드:", response.status_code)
7 print("응답 내용 일부:", response.text[:60])
8
```

실행 결과 (예시)

```
상태 코드: 200
응답 내용 일부: {"current_user_url": "https://api.github.com/user" ...}
```



★ 베스트 프랙티스

- 항상 가상환경에서 패키지를 설치하세요.
- 불필요한 패키지는 설치하지 마세요.
- requirements.txt로 의존성을 관리하세요.
- 프로젝트 문서에 필요한 패키지를 기록하세요.



기억할 키워드 pip, install, import, requirements.txt, 가상환경, 의존성 관리

내가 만든 패키지를 다른 사람들과 공유(배포)하고, 버전을 관리하는 방법을 알아봅니다.



Point

패키지를 체계적으로 배포하고 버전을 관리하면
다른 사람들도 쉽게 사용하고, 협업도 수월해집니다!

1. 패키지 배포란?

내가 만든 패키지를 PyPI(Python Package Index)에 업로드하여 다른 사람들이 pip로 설치할 수 있게 하는 과정입니다.



배포의 장점

- 전 세계 누구나 내 패키지를 쉽게 설치 가능
- 협업 및 재사용성 향상
- 버전 관리로 안정적인 배포 가능



사전 준비

- PyPI 계정 생성 : <https://pypi.org>
- 패키지 이름은 고유해야 합니다.
- twine 패키지 설치 : `pip install twine`



핵심 정리

- PyPI에 업로드하면 전 세계 누구나 pip로 설치할 수 있습니다.
- 빌드 → 업로드 → 설치 확인 순서로 진행합니다.
- 버전 관리를 통해 안정적이고 신뢰할 수 있는 패키지를 배포하세요.

2. 배포 절차

일반적인 패키지 배포 단계입니다.

① 패키지 구조 준비

`__init__.py`, 모듈 파일, `README.md`, `setup.py` 또는 `pyproject.toml` 등이 준비되어 있어야 합니다.

② 빌드(배포용 파일 생성)

```
python -m build
# dist/ 폴더에 배포 파일 생성
```

③ 업로드(테스트용 - 선택 사항)

```
twine upload --repository testpypi dist/*
# TestPyPI에 업로드하여 테스트
```

④ 업로드(실제 배포)

```
twine upload dist/*
# PyPI에 업로드
```

⑤ 설치 확인

```
pip install 내가-배포한-패키지-이름
```



실습 미션

1. 간단한 패키지(`simple_calc`)를 만들어 보세요.
2. 빌드 후 TestPyPI에 업로드하여 설치 테스트를 해보세요.
3. 실제 PyPI에 업로드하고, 다른 환경에서 설치해 보세요.

3. 예제: 간단한 패키지 배포

패키지 이름: `simple_calc`

배포 명령어 예시

```
1 # 1. 빌드
2 python -m build
3
4 # 2. 테스트 업로드 (선택)
5 twine upload --repository testpypi dist/*
6
7 # 3. 실제 업로드
8 twine upload dist/*
```

설치 예시 (사용자 입장)

```
$ pip install simple-calc
Collecting simple-calc
  Downloading simple_calc-0.1.0-py3-none-any.whl (5.2 kB)
Installing collected packages: simple-calc
Successfully installed simple-calc-0.1.0
```

☆ 버전 관리

`setup.py` 또는 `pyproject.toml`에 버전을 명시하고,
변경 시마다 버전을 올려서 배포합니다.
예) 0.1.0 → 0.1.1 → 0.2.0



도전 과제

- `README.md`를 작성하고 패키지 설명을 더 풍부하게 만들어 보세요.
- 버전을 올려(예: 0.1.1) 다시 배포해 보세요.
- 의존성 패키지가 있는 프로젝트를 배포해 보세요.

프로젝트마다 독립된 환경을 만들어 패키지 버전 충돌을 방지하고, 깔끔하게 관리해 봅시다.



Point

가상환경을 사용하면 프로젝트별로 필요한 패키지와 버전을 독립적으로 관리할 수 있습니다!

1. 가상환경이란?



가상환경(Virtual Environment)

독립된 Python 실행 환경을 만들어
프로젝트별로 다른 패키지와 버전을 사용 가능

☆ 왜 사용할까요?

- 프로젝트마다 다른 패키지 버전 필요
- 패키지 충돌 방지
- 시스템 Python 환경 보호
- 협업 시 동일한 환경 재현 용이



예시

프로젝트 A는 requests==2.25.1 사용,
프로젝트 B는 requests==2.31.0 사용
→ 각각의 가상환경에서 독립적으로 관리!



핵심 정리

- 가상환경은 독립된 Python 환경을 제공합니다.
- 프로젝트마다 다른 패키지와 버전을 안전하게 사용할 수 있습니다.
- requirements.txt를 활용하면 환경을 쉽게 공유할 수 있습니다.

2. 가상환경 생성 및 활성화



Windows

```
1 # 1. 가상환경 생성
2 python -m venv .venv          # .venv 폴더 생성
3
4 # 2. 가상환경 활성화
5 .venv\Scripts\activate        # 가상환경 활성화
6
7 # 3. 비활성화
8 deactivate                    # 가상환경 비활성화
```



macOS / Linux

```
1 # 1. 가상환경 생성
2 python3 -m venv .venv         # .venv 폴더 생성
3
4 # 2. 가상환경 활성화
5 source .venv/bin/activate     # 가상환경 활성화
6
7 # 3. 비활성화
8 deactivate                    # 가상환경 비활성화
```



활성화 확인

프롬프트 앞에 (.venv) 가 표시되면 활성화된 상태입니다!
(.venv) C:\myproject> 또는 (.venv) \$

3. 가상환경에서 패키지 관리

```
1 # 패키지 설치
2 pip install requests pandas    # 필요한 패키지 설치
3
4 # 설치된 패키지 확인
5 pip list                      # 설치된 패키지 목록 확인
6
7 # 패키지 버전 고정 파일 생성
8 pip freeze > requirements.txt  # 현재 환경의 패키지 목록 저장
9
10 # requirements.txt로 설치
11 pip install -r requirements.txt # 다른 환경에서 동일하게 설치
```

4. 가상환경 구조 예시

```
myproject/
├── .venv/          ← 가상환경 폴더
│   ├── Scripts/   (Windows) 또는 bin/ (macOS/Linux)
│   ├── Lib/       (Windows) 또는 lib/ (macOS/Linux)
│   └── pyenv.cfg
├── app.py
└── requirements.txt ← 패키지 목록 파일
```

pyenv.cfg 예시

```
home = C:\Python39
include-system-site-packages = false
version = 3.9.6
...
```



실습 미션

1. myproject 폴더를 만들고 가상환경(.venv)을 생성해 보세요.
2. 가상환경을 활성화하고 requests, pandas를 설치해 보세요.
3. pip list로 설치 확인 후, pip freeze > requirements.txt 실행해 보세요.



도전 과제

- 다른 컴퓨터에서 requirements.txt로 동일 환경을 만들어 보세요.
- 가상환경을 여러 개 만들어 프로젝트별로 관리해 보세요.
- pyproject.toml과 poetry를 이용한 환경 관리도 찾아보세요!

만든 패키지를 PyPI에 업로드하여 다른 사람들이 쉽게 설치하고 사용할 수 있도록 배포해 봅시다.



Point

PyPI에 배포하면 전 세계 개발자들이
내 패키지를 pip install로 사용할 수 있습니다!

1. 배포 전체 흐름

패키지 배포는 다음 순서로 진행됩니다.



☆ 필수 도구 설치

아래 도구들이 필요합니다.

- build : 패키지를 배포용 파일로 빌드
- twine : PyPI에 업로드
- (선택) keyring : 인증 정보 안전 관리

```
pip install --upgrade build twine keyring
```

⚠ 주의사항

- PyPI 계정이 필요합니다. (<https://pypi.org>)
- 업로드할 때는 API Token을 사용하는 것을 권장합니다.
- 한 번 업로드한 버전은 덮어쓸 수 없습니다.
(버전은 항상 새로 올려야 합니다.)

🔗 핵심 정리

- python -m build 로 배포용 파일을 생성합니다.
- twine upload 로 PyPI에 업로드합니다.
- API Token을 사용하면 더 안전하게 배포할 수 있습니다.
- 업로드 후 pip install 로 설치 테스트를 꼭 해보세요.

2. 배포용 파일 빌드

프로젝트 루트에서 다음 명령으로 배포용 파일을 만듭니다.

```
# 이전 빌드 결과 삭제 (선택)
rm -rf dist/ build/ *.egg-info

# 배포용 파일 빌드 (sdist, wheel 생성)
python -m build
```

✅ 생성되는 파일

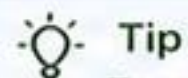
- dist/ 폴더에 다음 파일들이 생성됩니다.
- yourpkg-0.1.0.tar.gz (Source Distribution)
- yourpkg-0.1.0-py3-none-any.whl (Wheel)

3. 업로드 (TestPyPI 먼저 권장)

실제 PyPI 업로드 전에 TestPyPI로 테스트해 보세요.

```
# TestPyPI 업로드
python -m twine upload --repository testpypi dist/*

# (정상 업로드 확인 후) 실제 PyPI 업로드
python -m twine upload dist/*
```



Tip

TestPyPI : <https://test.pypi.org>
실제 PyPI : <https://pypi.org>



실습 미션

1. 패키지를 빌드하여 dist/ 폴더를 생성해 보세요.
2. TestPyPI에 업로드해 보고 설치 테스트를 해보세요.
3. 실제 PyPI에 업로드하고, 새 환경에서 설치해 보세요.

4. 인증 방법 (API Token 사용 권장)

- 1) PyPI에서 API Token 생성 (계정 → API tokens)
- 2) ~/.pypirc 파일 생성

```
[distutils]
index-servers = pypi

[pypi]
username = __token__
password = pypi-..... (발급받은 토큰)

[testpypi]
repository = https://test.pypi.org/legacy/
username = __token__
password = pypi-..... (TestPyPI 토큰)
```

i 권장: keyring 사용 (더 안전)

keyring을 사용하면 비밀번호를 파일에 저장하지 않습니다.

```
python -m keyring set
--service pypi
--username __token__
--password <your-token>
```

5. 사용자 설치 (확인)

업로드가 완료되면 다른 환경에서 설치 테스트!

```
pip install yourpkg
```

```
>>> import yourpkg
>>> yourpkg.__version__
'0.1.0'
```

💡 도전 과제

- 배포 자동화를 위한 GitHub Actions 워크플로우를 만들어 보세요.
- README와 예제 코드를 추가하여 더 완성도 높은 패키지를 만들어 보세요.
- 다른 사람의 피드백을 받아 패키지를 개선해 보세요!

작성한 패키지를 PyPI에 공개하여 다른 사람들이 쉽게 설치하고 사용할 수 있도록 배포합니다.



Point

PyPI에 패키지를 공개하면 전 세계 개발자들이
내 패키지를 손쉽게 사용할 수 있습니다!

1. 사전 준비

PyPI에 배포하기 위한 준비를 합니다.



필수 조건

- PyPI 계정 보유 (<https://pypi.org>)
- 패키지 버전이 올바르게 설정되어 있음
- README.md, LICENSE 등 문서 준비



권장 사항

- 패키지 설명 (long_description) 작성
- Trove Classifiers 추가
- 테스트 코드 및 예제 포함

2. 빌드 파일 생성

소스 배포용(sdist)과 배포용 휠(wheel) 파일을 생성합니다.

```
$ cd myproject
$ python -m build

* Creating isolated environment: done.
* Building sdist...
* Building wheel...
Successfully built myproject-0.1.0.tar.gz
Successfully built myproject-0.1.0-py3-none-any.whl
```



Tip

build 모듈은 빌드 환경을 자동으로 구성해 줍니다.
(setup.py 기반 또는 pyproject.toml 기반 모두 지원)

3. 배포 테스트 (TestPyPI)

실제 PyPI에 배포하기 전에 TestPyPI에 먼저 업로드하여 테스트합니다.

```
$ python -m twine upload --repository testpypi dist/*

Uploading distributions to https://test.pypi.org/legacy/
Uploading myproject-0.1.0-py3-none-any.whl
Uploading myproject-0.1.0.tar.gz

View at:
https://test.pypi.org/project/myproject/0.1.0/
```



TestPyPI란?

- PyPI와 동일한 환경의 테스트 저장소입니다.
- 실제 PyPI에 영향을 주지 않고 배포 과정을 확인할 수 있습니다.

4. 실제 PyPI에 배포

테스트가 완료되면 실제 PyPI에 배포합니다.

```
$ python -m twine upload dist/*

Uploading distributions to https://upload.pypi.org/legacy/
Uploading myproject-0.1.0-py3-none-any.whl
Uploading myproject-0.1.0.tar.gz

View at:
https://pypi.org/project/myproject/0.1.0/
```



베스트 프랙티스

- 버전은 PEP 440을 따르세요. (예: 0.1.0, 1.0.0)
- 패키지 설명을 명확하고 상세하게 작성하세요.
- 정기적으로 버그 수정 및 기능 개선 버전을 배포하세요.

5. 설치 확인 (다른 환경에서)

다른 가상환경이나 다른 컴퓨터에서 설치를 확인합니다.

```
$ pip install myproject
```

```
>>> import myproject
>>> myproject.__version__
'0.1.0'
```



주의사항

- 한 번 업로드한 파일은 덮어쓸 수 없습니다.
- 실수로 잘못된 파일을 올리면 버전을 올려 새로 배포해야 합니다.
- 중요한 정보(API Token 등)는 절대 공개 저장소에 올리지 마세요.

6. 배포 후 체크리스트

- ✓ PyPI 프로젝트 페이지 확인
(<https://pypi.org/project/내패키지이름/>)
- ✓ 패키지 설명, 파일, 버전 정보가 올바르게 표시되는지 확인
- ✓ 다른 환경에서 설치 및 사용 테스트
- ✓ README의 사용 예제 정상 동작 확인
- ✓ 필요 시 GitHub, 블로그 등에서 홍보!



기억할 키워드

build, twine, upload, TestPyPI, PyPI,
long_description, version, Trove Classifiers

18 패키지 사용하기 (설치한 패키지 활용)

설치한 패키지를 내 프로젝트에서 import하여 사용해 봅시다.



Point

패키지 설치의 시작일 뿐입니다!

이제 실제로 import하고 활용하는 방법을 익혀 봅시다.

1. 패키지 import하기

설치한 패키지를 내 코드에서 import합니다.

example.py

```
1 import requests          # requests 패키지 import
2 import pandas as pd       # pandas 패키지 import
3 from datetime import datetime  # 특정 모듈/클래스 import
4
```

2. 패키지 기능 사용하기

import한 패키지의 기능을 사용합니다.

example.py

```
1 import requests
2
3 # GitHub API 호출
4 url = "https://api.github.com/repos/python/cpython"
5 response = requests.get(url)
6 data = response.json()
7 print(data["full_name"])  # 출력: python/cpython
```

3. 자주 사용하는 패턴

자주 사용하는 패턴들을 익혀두세요.

별칭 사용 (as)

```
import numpy as np
# np.array([1, 2, 3]) 처럼 사용
```

특정 기능만 import

```
from math import sqrt, pow
# sqrt(16), pow(2, 3) 처럼 사용
```

전체 모듈 import

```
import datetime
# datetime.datetime.now() 처럼 사용
```

상대 import (패키지 내부에서)

```
from .utils import helper
from ..core import config
```

4. 예제: 다양한 패키지 활용

여러 패키지를 조합하여 활용해 봅시다.

data_analysis.py

```
1 import pandas as pd
2 import requests
3 from datetime import datetime
4
5 # 1. 데이터 수집
6 res = requests.get("https://api.github.com/repos/pandas-dev/pandas")
7 data = res.json()
8
9 # 2. 데이터 가공 및 분석
10 df = pd.DataFrame([data])
11 df['fetched_at'] = datetime.now()
12 print(df[['full_name', 'stargazers_count', 'fetched_at']])
```

실행 결과 예시

```
full_name  stargazers_count  fetched_at
0 pandas-dev/pandas        54211  2024-05-20 14:30:25.123456
```



확인 사항

- import 문을 파일 상단에 작성하는 것이 일반적입니다.
- 패키지 문서(README, 공식 문서)를 참고하세요.
- 함수/클래스 사용법과 매개변수를 확인하세요.
- 예제 코드를 직접 실행해보며 이해하세요.



예러 해결 팁

- ModuleNotFoundError: 패키지 설치 확인
- ImportError: import 경로 및 이름 확인
- pip list로 설치된 패키지 목록 확인
- 가상환경이 활성화되어 있는지 확인



실습 아이디어

- requests로 날씨 정보를 가져와 출력해 보세요.
- pandas로 CSV 파일을 읽고 요약 정보를 확인해 보세요.
- matplotlib로 간단한 그래프를 그려보세요.
- 여러 패키지를 조합해 작은 프로젝트를 만들어 보세요.



도전 과제

설치한 패키지들을 활용하여
"GitHub 인기 저장소 분석 대시보드"
를 만들어 보세요!
(데이터 수집 → 분석 → 시각화)



가상환경을 만들고 패키지를 설치, 관리, 제거하는 실습을 통해 전체 흐름을 익혀봅시다.



Point

실습을 통해 가상환경과 패키지 관리 과정을 직접 경험하면 더 오래 기억할 수 있습니다!

1. 실습 목표



- 가상환경 생성 및 활성화
- 패키지 설치 및 버전 확인
- 패키지 목록 관리
- 패키지 제거 및 정리

2. 실습 순서도



3. 실습 단계별 명령어

① 가상환경 생성

```
python -m venv .venv
```

.venv 폴더 생성

② 가상환경 활성화

- Windows
`.venv\Scripts\activate` # PowerShell 또는 CMD
- macOS / Linux
`source .venv/bin/activate` # 터미널

③ 패키지 설치

```
pip install requests pandas
```

④ 설치된 패키지 목록 확인

```
pip list
```

⑤ 특정 패키지 정보 확인

```
pip show requests
```

⑥ 패키지 제거

```
pip uninstall requests
```

⑦ 나가기 및 정리

```
deactivate
```

(선택) `rm -rf .venv` (macOS/Linux)
 # `rmdir /s .venv` (Windows)

4. 실행 예시 (Windows)

```
PS C:\myproject> python -m venv .venv
PS C:\myproject> .\.venv\Scripts\activate
(.venv) PS C:\myproject> pip install requests pandas
....
Successfully installed certifi charset-normalizer idna numpy
pandas python-dateutil pytz requests six urllib3
(.venv) PS C:\myproject> pip list
Package      Version
-----
pandas       2.2.2
requests     2.32.3
(... 생략 ...)
(.venv) PS C:\myproject> pip uninstall requests
Found existing installation: requests 2.32.3
Uninstalling requests-2.32.3:
  Successfully uninstalled requests-2.32.3
(.venv) PS C:\myproject> deactivate
PS C:\myproject>
```

5. 모범 사례

- ✓ 프로젝트마다 별도의 가상환경 사용
- ✓ 필요한 패키지만 설치
- ✓ requirements.txt로 의존성 관리
- ✓ 주기적으로 패키지 업데이트 확인
- ✓ 사용하지 않는 패키지는 제거
- ✓ 가상환경 폴더(.venv)는 .gitignore에 추가

☆ 알아두기

- 가상환경은 프로젝트 폴더 안에 만드는 것이 일반적입니다.
- 활성화된 가상환경에서 설치한 패키지는 해당 환경에만 적용됩니다.
- deactivate 명령으로 언제든지 나갈 수 있습니다.



Tip

pip install 패키지명==버전 을 사용하면
 특정 버전으로 설치할 수 있습니다.
 예) pip install pandas==2.2.2



참고

pip freeze > requirements.txt
 명령으로 현재 설치된 패키지 목록을 파일로 저장하고,
 다른 환경에서 pip install -r requirements.txt 로
 동일한 환경을 재현할 수 있습니다.



주의

(venv) 프롬프트가 보이지 않으면
 가상환경이 활성화되지 않은 것입니다.
 반드시 활성화 후 작업하세요!



실습 미션

1. myenv 라는 가상환경을 만들고 활성화하세요.
2. requests, numpy 패키지를 설치하세요.
3. 설치 목록을 확인하고, numpy를 제거하세요.
4. 가상환경을 비활성화하고 정리하세요.

Day 2에서 학습한 가상환경(.venv)과 패키지 관리 내용을 핵심만 정리해 봅시다.



Point

가상환경을 만들고, 패키지를 설치·관리하는 흐름을 정확히 이해하면 어떤 프로젝트도 문제없이 시작할 수 있습니다!

1. 핵심 개념



가상환경 (venv)

프로젝트마다 독립된 Python 환경을 제공하여 패키지 충돌을 방지합니다.



패키지 (Package)

다른 개발자가 만든 기능(모듈)의 집합으로, PyPI에서 쉽게 설치하여 사용할 수 있습니다.



패키지 관리 도구 (pip)

PyPI에서 패키지를 검색, 설치, 삭제, 업그레이드 할 수 있게 해주는 도구입니다.

2. 가상환경 명령어 요약

생성

```
python -m venv .venv
```

활성화

• Windows (PowerShell 또는 CMD)

```
.venv\Scripts\activate
```

• macOS / Linux

```
source .venv/bin/activate
```

비활성화

```
deactivate
```

3. 패키지 관리 명령어 요약

설치

```
pip install <패키지명>
```

패키지를 설치합니다.

목록 확인

```
pip list
```

설치된 패키지 목록을 확인합니다.

특정 패키지 정보 확인

```
pip show <패키지명>
```

패키지의 상세 정보를 확인합니다.

업그레이드

```
pip install --upgrade <패키지명>
```

패키지를 최신 버전으로 업그레이드합니다.

삭제

```
pip uninstall <패키지명>
```

패키지를 제거합니다.

의존성 관리

```
pip install -r requirements.txt
```

requirements.txt에 적힌 패키지를 설치합니다.

의존성 저장

```
pip freeze > requirements.txt
```

현재 환경의 패키지 목록을 파일로 저장합니다.

4. 프로젝트 흐름 전체 요약



1 가상환경 생성

```
python -m venv .venv
```



2 가상환경 활성화

```
.venv\Scripts\activate (Windows)  
source .venv/bin/activate (macOS/Linux)
```



3 패키지 설치

```
pip install <패키지명>
```



4 작업 진행

프로젝트 개발 및 실행



5 환경 저장/공유

```
pip freeze > requirements.txt
```



6 환경 재현

```
pip install -r requirements.txt
```

5. 의존성 관리 예시

의존성 저장

```
pip freeze > requirements.txt
```

예시 (requirements.txt)

```
requests==2.32.3  
pandas==2.2.2  
numpy==1.26.4
```

의존성 설치 (다른 컴퓨터)

```
pip install -r requirements.txt
```

결과

- ✓ requirements.txt에 있는 모든 패키지 설치
- ✓ 동일한 버전으로 환경 재현 가능

6. 가상환경 구조 예시

myproject/

```
.venv/ ← 가상환경 폴더  
├── Scripts/ (Windows) 또는 bin/ (macOS/Linux)  
├── Lib/ (Windows) 또는 lib/ (macOS/Linux)  
├── pyenv.config  
├── app.py  
└── requirements.txt ← 패키지 목록 파일
```

pyenv.config 예시

```
home = C:\Python39  
include-system-site-packages = false  
version = 3.9.6  
...
```

7. 자주 발생하는 문제

- ❗ **ModuleNotFoundError**
→ 패키지가 설치되지 않았거나 다른 환경에서 설치됨
→ 가상환경이 활성화되어 있는지 확인!
- ❗ **pip 명령이 인식되지 않음**
→ Python 또는 pip가 PATH에 추가되지 않았을 수 있음
- ❗ **패키지 버전 충돌**
→ 가상환경을 사용하면 각 프로젝트별로 독립 관리 가능!



핵심 정리

- 가상환경은 프로젝트별 독립된 환경을 제공합니다.
- 패키지는 PyPI에서 쉽게 설치하고 관리할 수 있습니다.
- requirements.txt를 활용하면 환경을 간편하게 공유하고 재현할 수 있습니다.



실습 미션

1. 새 가상환경을 만들고 활성화하세요.
2. requests, pandas, numpy를 설치하세요.
3. pip list로 목록을 확인하고, pip freeze > requirements.txt로 저장하세요.
4. 가상환경을 비활성화한 뒤, 다시 requirements.txt로 환경을 재현해 보세요.



도전 과제

- 다른 컴퓨터에서 requirements.txt로 동일 환경을 만들어 보세요.
- pyproject.toml과 poetry를 이용한 패키지 관리도 탐색해 보세요!
- GitHub Actions 워크플로우로 자동 환경 구축을 시도해 보세요!